

Lecture 10: Verification of Neural Networks (Cont.)

CS 580B1: Trustworthy Machine Learning
Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

[Trustworthy AI Systems](#) at UIUC by Gagandeep Singh and Madhu Parthasarathy

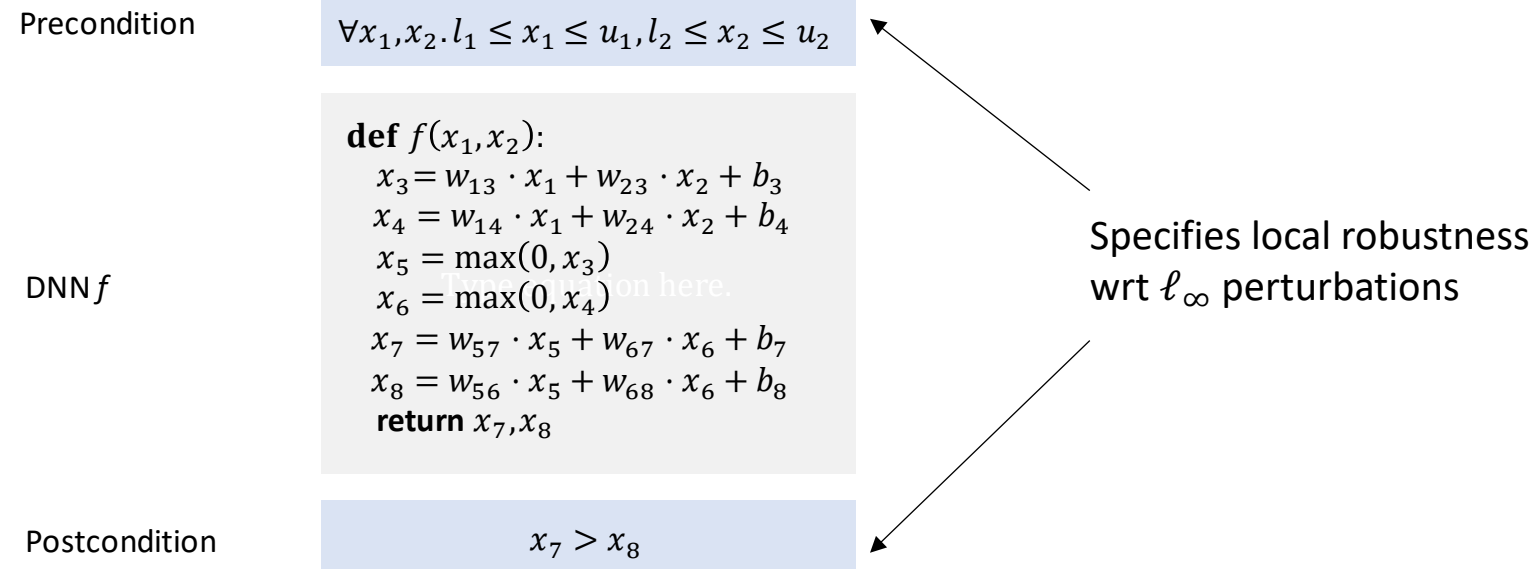
Other relevant courses on Trustworthy Machine Learning:

[CS 329T](#) at Stanford

[CS 521](#) at UIUC

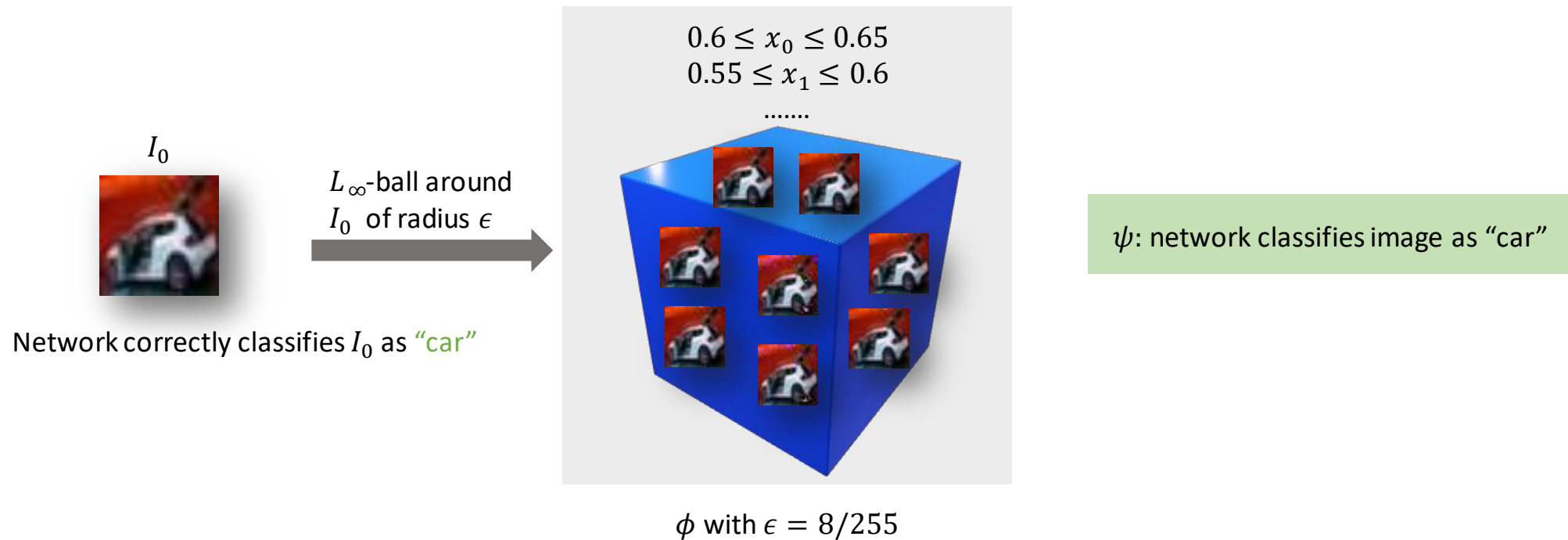
[ECE1784H/CSC2559H](#) at U Toronto

Recap: Formal specifications for DNNs

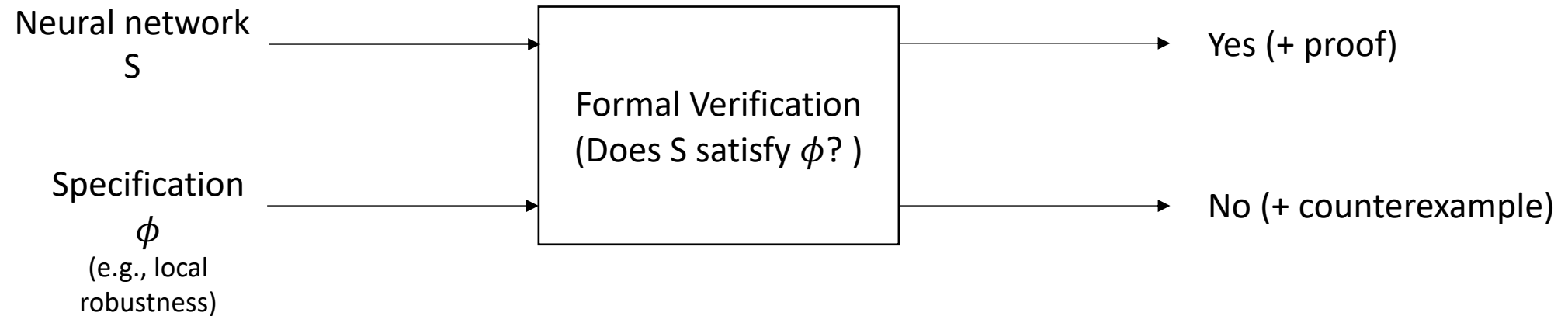


Either prove that the network output satisfies the postcondition for all inputs in the pre-condition or find a counterexample

Recap: Formal specifications for DNNs--Local robustness



Recap: Formal verification of DNNs



Recap: Techniques for Formal verification of DNNs

Incomplete	Abstract interpretation: Box , Zonotope, DeepPoly
Complete	Mixed Integer Linear Programming (MILP) SMT solvers (Reluplex)

Active area of research with annual competition: VNNComp
Current winner: alpha-beta crown

Recap: DNN verification as constraint solving

Precondition

$$\forall x_1, x_2. l_1 \leq x_1 \leq u_1, l_2 \leq x_2 \leq u_2$$

DNN f

```
def f(x1, x2):  
    x3 = w13 · x1 + w23 · x2 + b3  
    x4 = w14 · x1 + w24 · x2 + b4  
    x5 = max(0, x3)  
    x6 = max(0, x4)  
    x7 = w57 · x5 + w67 · x6 + b7  
    x8 = w56 · x5 + w68 · x6 + b8  
    return x7, x8
```

Postcondition

$$x_7 > x_8$$

- Set of variables real-valued variables V
- Atomic predicates:
 - Linear inequalities of the form $\sum_{v_i \in V} a_i v_i \leq c_i$
 - ReLU equation of the form $v_i = \text{ReLU}(v_j)$
- DNN represented by a conjunction of atomic predicates $\psi_N = \psi_1 \wedge \cdots \wedge \psi_t$
- Precondition ψ_{pre} and postcondition ψ_{post}
- Check satisfiability of $\psi_{pre} \wedge \psi_N \wedge \neg \psi_{post}$

Recap: Reluplex

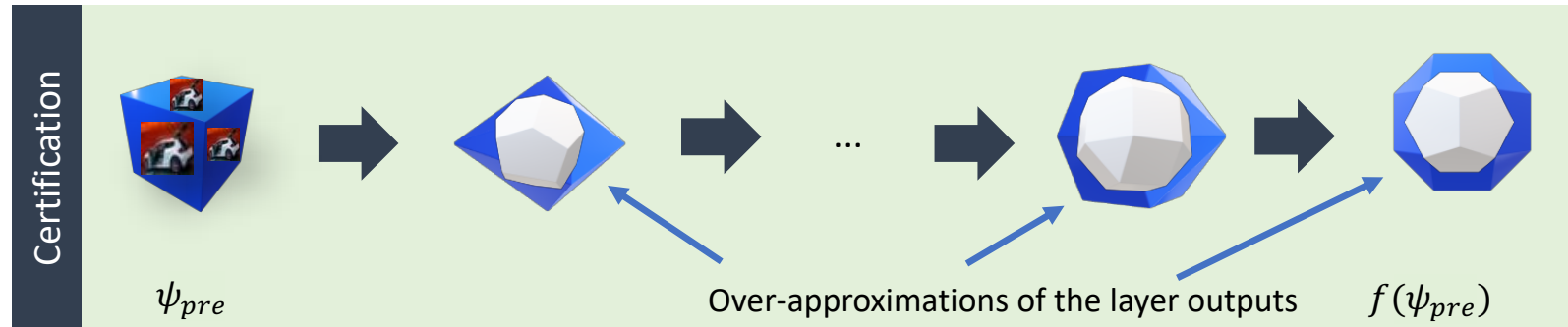
- Proposed in paper “Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks” by Katz et al. in CAV’17
- SMT-solver for quantifier-free linear real arithmetic + ReLUs
- Based on the *Simplex* method for checking satisfiability of conjunctions of linear real arithmetic constraints
 - *Simplex* + ReLUs = Reluplex
 - Applicable to other piece-wise linear functions
- Sound and complete!

Formal Security Analysis of Neural Networks using
Symbolic Intervals by Wang et al

Sound but incomplete methods

We will investigate a specific type of incomplete method, based on bound propagation through the neural network. Starting with the initial pre-condition ϕ , we will “pass” ϕ through the network, computing a convex over-approximation of the effect of each layer on ϕ . Next, let's look at the “recipe” for certification with bound propagation

Step 1: compute convex $f(\psi_{pre})$ by propagating ψ_{pre}

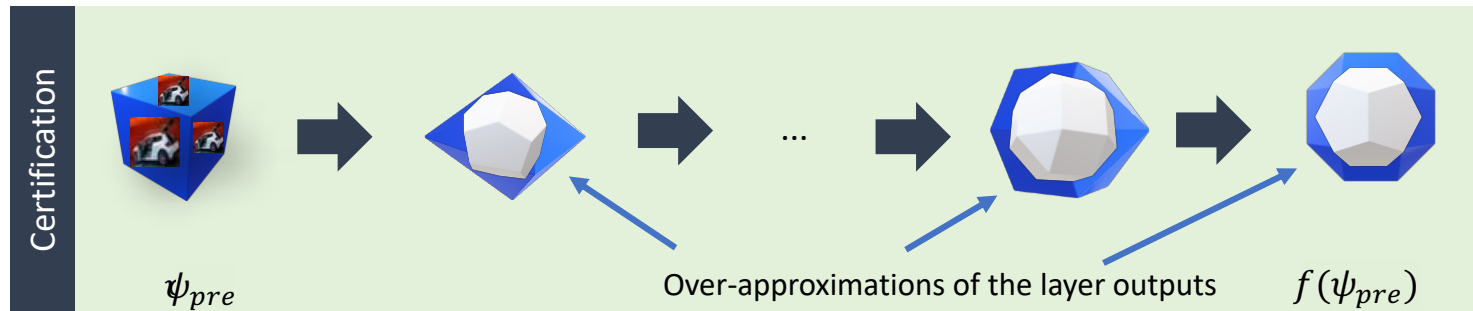


ψ_{pre}
 $x_0 = [0.588, 0.65]$
 $x_1 = [0.545, 0.608]$
 $x_2 = [0.533, 0.596]$
 $\dots$
 $x_{3071} = [0.4, 0.463]$

All possible outputs

$f(\psi_{pre})$
 $o_0 = 0$
 $o_1 = 2.60 + 0.015\eta_0 + 0.023\eta_1 + 5.181\eta_2 + \dots$
 $o_2 = 4.63 - 0.005\eta_0 - 0.006\eta_1 + 0.023\eta_2 + \dots$
 $\dots$
 $o_9 = 0.12 - 0.125\eta_0 + 0.102\eta_1 + 3.012\eta_2 + \dots$
 $\forall i. \eta_i \in [0, 1]$

Step 2: verify ψ_{post}



$f(\psi_{pre})$

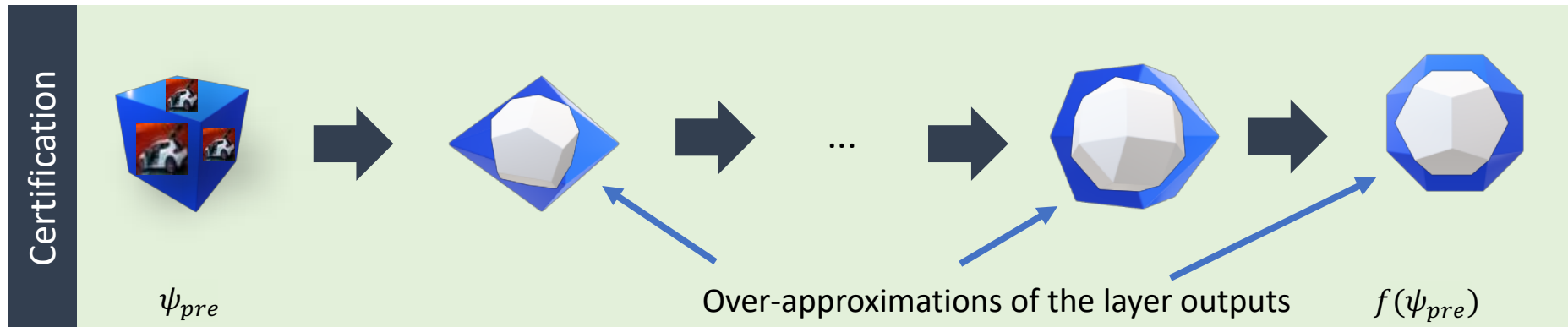
$$\begin{aligned} o_0 &= 0 \\ o_1 &= 2.60 + 0.015\eta_0 + 0.023\eta_1 + 5.181\eta_2 + \dots \\ o_2 &= 4.63 - 0.005\eta_0 - 0.006\eta_1 + 0.023\eta_2 + \dots \\ &\dots \\ o_9 &= 0.12 - 0.125\eta_0 + 0.102\eta_1 + 3.012\eta_2 + \dots \\ \forall i. \eta_i &\in [0,1] \end{aligned}$$

ψ_{post} : every point in ϕ classifies as car

ψ_{post} : $\forall l \neq car. o_{car} > o_l$

$f(\psi_{pre})$ is an **over-approximation**, hence if we fail to prove ψ_{post} could be the property is actually violated or there was over-approximation introduced during propagation which prohibits provability.

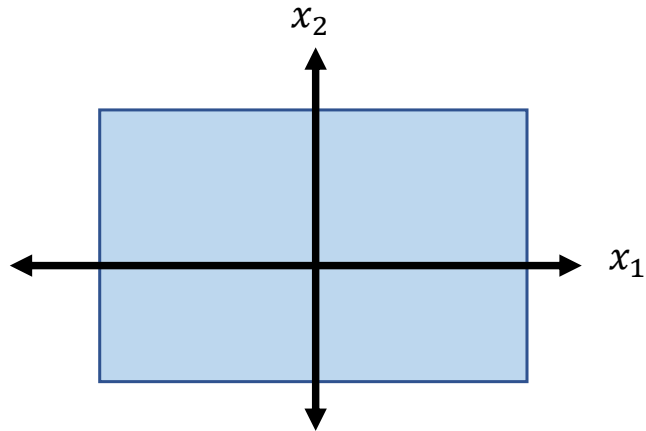
Key challenge: How to propagate convex shapes?



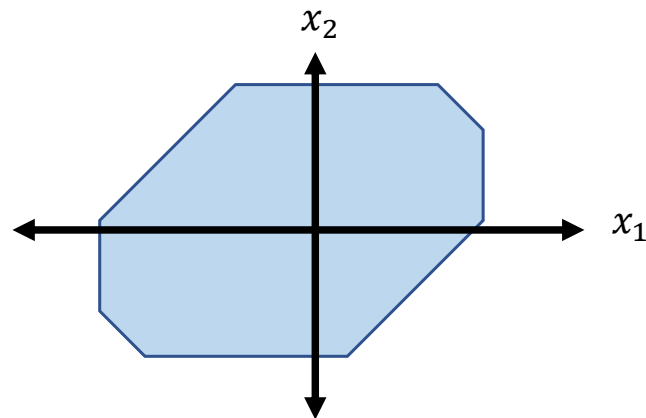
To instantiate incomplete methods which use bound propagation, we need two parts:

1. What is the convex approximation  ? E.g., Box, Zonotope, Polyhedra
2. How are these convex approximations produced? That is, what is the effect of the layer  on a given approximation  ? This effect is often called an **abstract transformer** as it transforms abstract shapes.

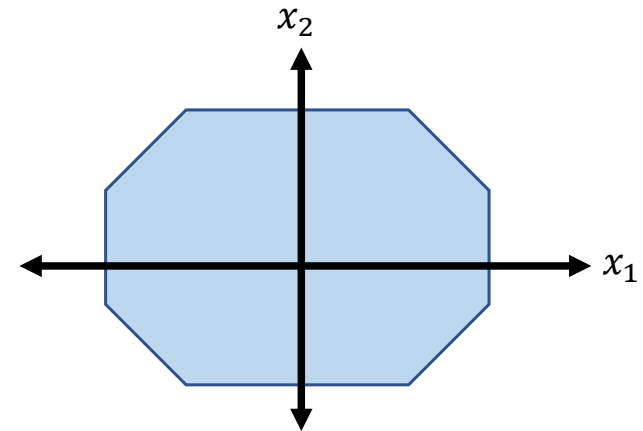
Popular convex shapes



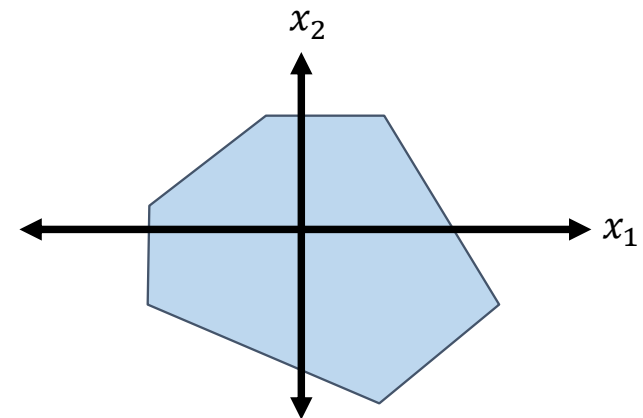
Box: $l_i \leq x_i \leq u_i$



Zonotope: $\hat{x}_i = \alpha_0 + \sum_i \alpha_i \epsilon_i, \epsilon_i \in [-1, 1]$



Octagon: $l_i \leq x_i \leq u_i, \pm x_i \pm x_j \leq c_{ij}$



Polyhedra: $\sum_i a_i x_i \leq c$

Speed vs. precision tradeoff

A trade-off between approximation and speed exists:

transformers for Box are fast, but imprecise,

while Polyhedra is more precise but has exponential complexity.

Box abstract transformer: Addition

$$[\mathbf{a}, \mathbf{b}] +^\# [\mathbf{c}, \mathbf{d}] = [\mathbf{a} + \mathbf{c}, \mathbf{b} + \mathbf{d}] \quad \text{Addition transformer}$$

- Here, $\mathbf{a}, \mathbf{b} \in \mathbf{R}^m$ where $\forall i. a_i \leq b_i$
- $^\#$ denotes the abstract effect of the operation on the box

Example in 1-dim (interval addition) $[1, 3] +^\# [5, 8] = [6, 11]$

Box abstract transformers for ReLU networks

$$[a, b] +^{\#} [c, d] = [a + c, b + d]$$

Addition transformer

$$-^{\#}[a, b] = [-b, -a]$$

Negation transformer

$$\mathbf{ReLU}^{\#}[a, b] = [\mathbf{ReLU}(a), \mathbf{ReLU}(b)]$$

ReLU transformer

$$\lambda^{\#}[a, b] = [\lambda * a, \lambda * b]$$

Multiplication by a constant $\lambda > 0$

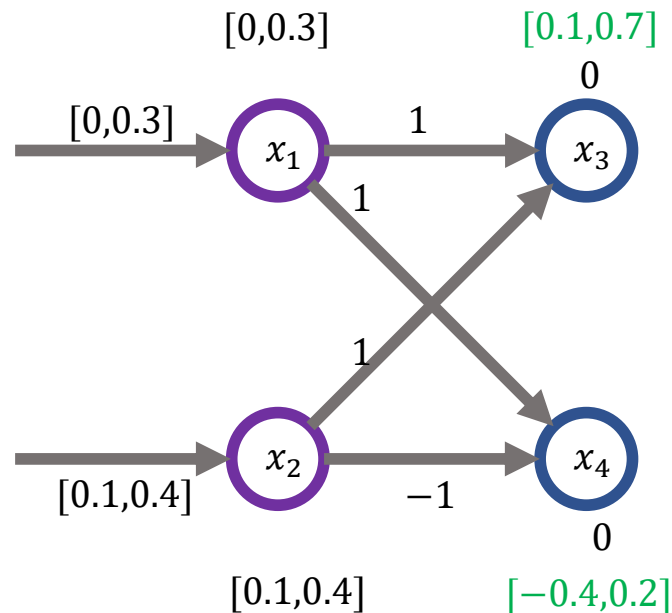
$$[a, b] >^{\#} [c, d] \text{ holds iff } a > d$$

Comparison transformer

- Here, $a, b \in \mathbf{R}^m$ where $\forall i. a_i \leq b_i$
- $\mathbf{ReLU}(x) = \max(0, x)$
- $^{\#}$ denotes the abstract effect of the operation on the box

Computation with box transformer

We have 2 pixels (x_1, x_2) as input ranging over $[0, 0.3]$ and $[0.1, 0.4]$



```
def f1(x1, x2):  
    x3 = x1 + x2  
    x4 = x1 - x2  
    return x3, x4
```

Bounds using Box:

$$0.1 \leq x_3 \leq 0.7$$

$$-0.4 \leq x_4 \leq 0.2$$

Exact bounds would be:

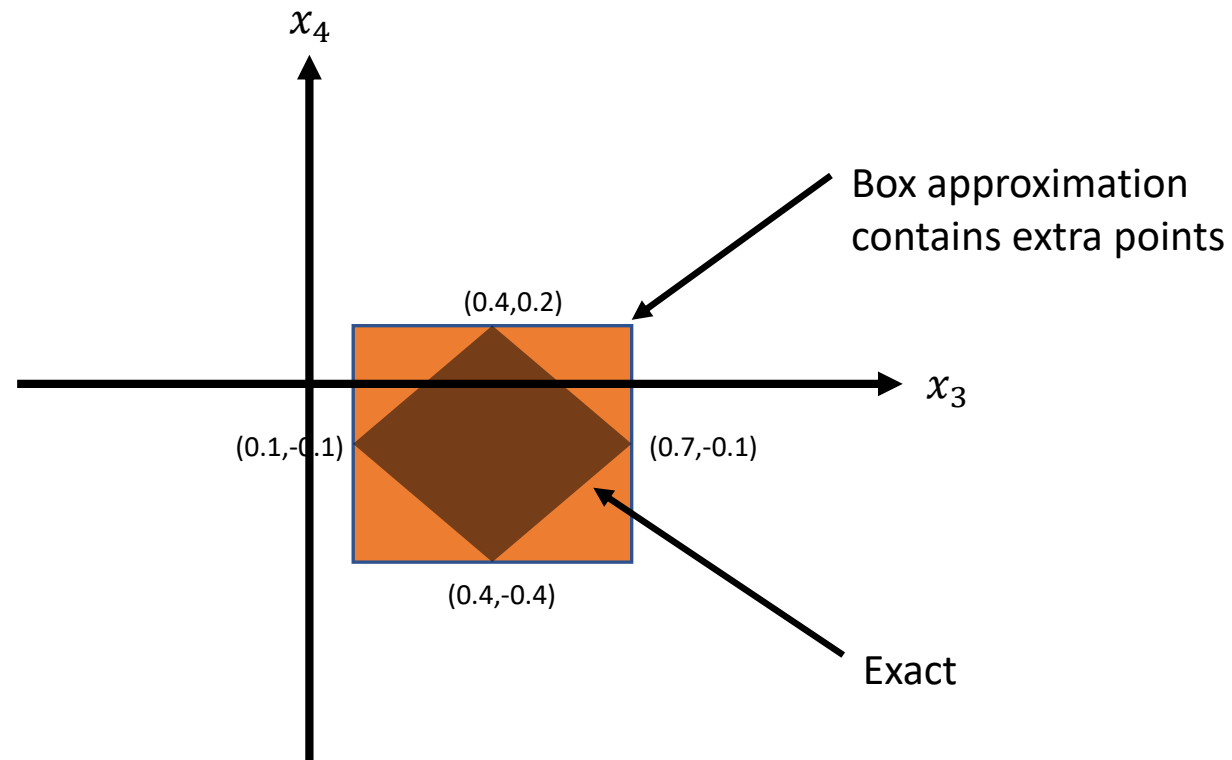
$$x_3 = x_1 + x_2$$

$$x_4 = x_1 - x_2$$

$$0 \leq x_1 \leq 0.3$$

$$0.1 \leq x_2 \leq 0.4$$

Optimal box transformer is not exact

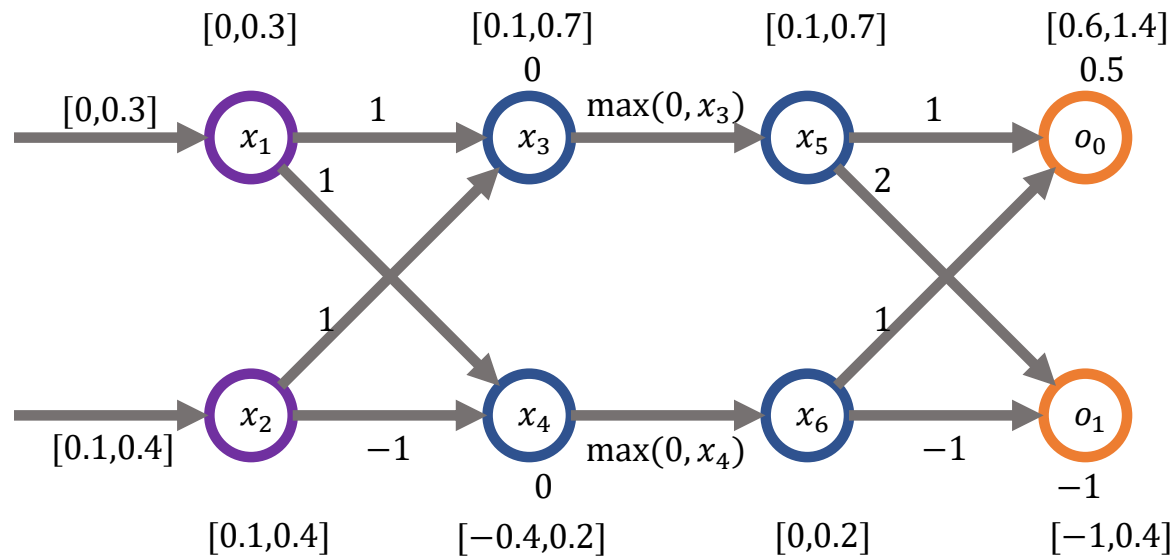


Key point: soundness and completeness

Even though the Box abstract transformer for the affine computation is optimal for the Box relaxation, it may not be complete (exact)! Nonetheless, even if not exact, it may be enough to verify the property of interest.

Box succeeds in verifying local robustness

We have 2 pixels (x_1, x_2) as input ranging over $[0, 0.3]$ and $[0.1, 0.4]$, we want to prove that $o_0 > o_1$ holds for all inputs

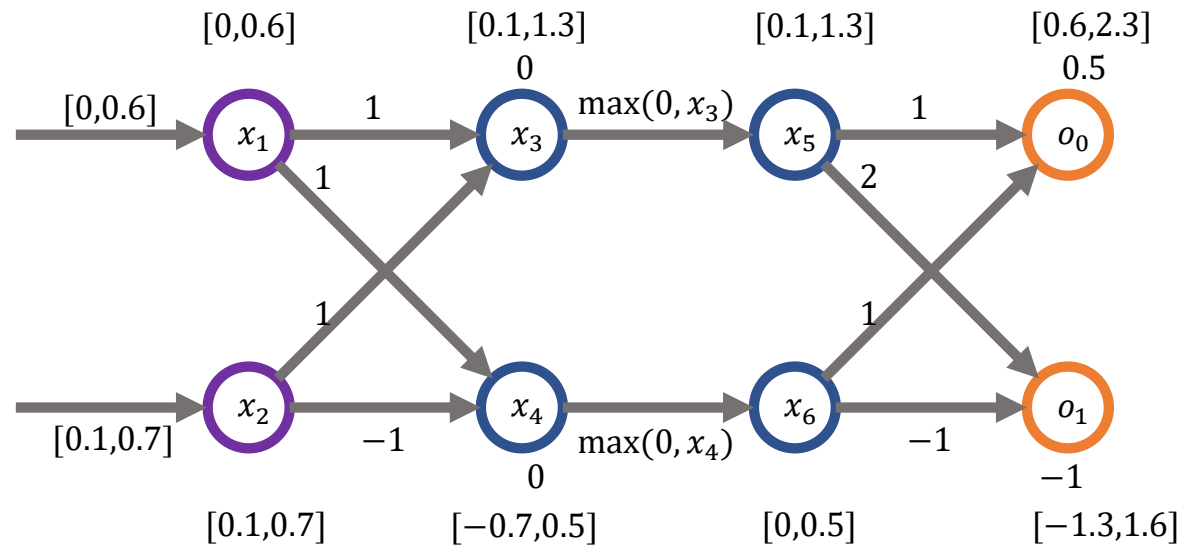


```
def f(x1, x2):  
    x3 = x1 + x2  
    x4 = x1 - x2  
    x5 = max(0, x3)  
    x6 = max(0, x4)  
    o0 = x5 + x6 + 0.5  
    o1 = 2 · x5 - x6 - 1  
    return o0, o1
```

Using Box, we succeed in proving the network classifies any input in the range as 0.
This is because $[0.6, 1.4] > [-1, 0.4]$, provably so.

Box fails in verifying local robustness

Let us **slightly increase** the range of the input pixels to $[0, 0.6]$ and $[0.1, 0.7]$



```
def f(x1, x2):  
    x3 = x1 + x2  
    x4 = x1 - x2  
    x5 = max(0, x3)  
    x6 = max(0, x4)  
    o0 = x5 + x6 + 0.5  
    o1 = 2 · x5 - x6 - 1  
    return o0, o1
```

Using Box, we failed to prove the network classifies any input in the range as 0, even though property actually holds. This is because $[0.6, 2.3]$ is not $> [-1.3, 1.6]$, provably so.

Optimizations in the paper

- Propagate symbolic intervals that can get concretized at ReLU nodes
- Iterative interval refinement
 - Influence analysis
 - Monotonicity

DNN verification

- Instead of Box domain, one can use Zonotopes to get reasonably scalable and more precise approximations
- Constraint solving and abstract domains can be combined in different ways
- Local robustness verification can be integrated into training
 - Parameters are updated not only to minimize loss but also to ensure that a verification procedure (e.g. based on Box abstract domain) gives robustness guarantee